

情報システム工学実験

AIプログラミング (3)

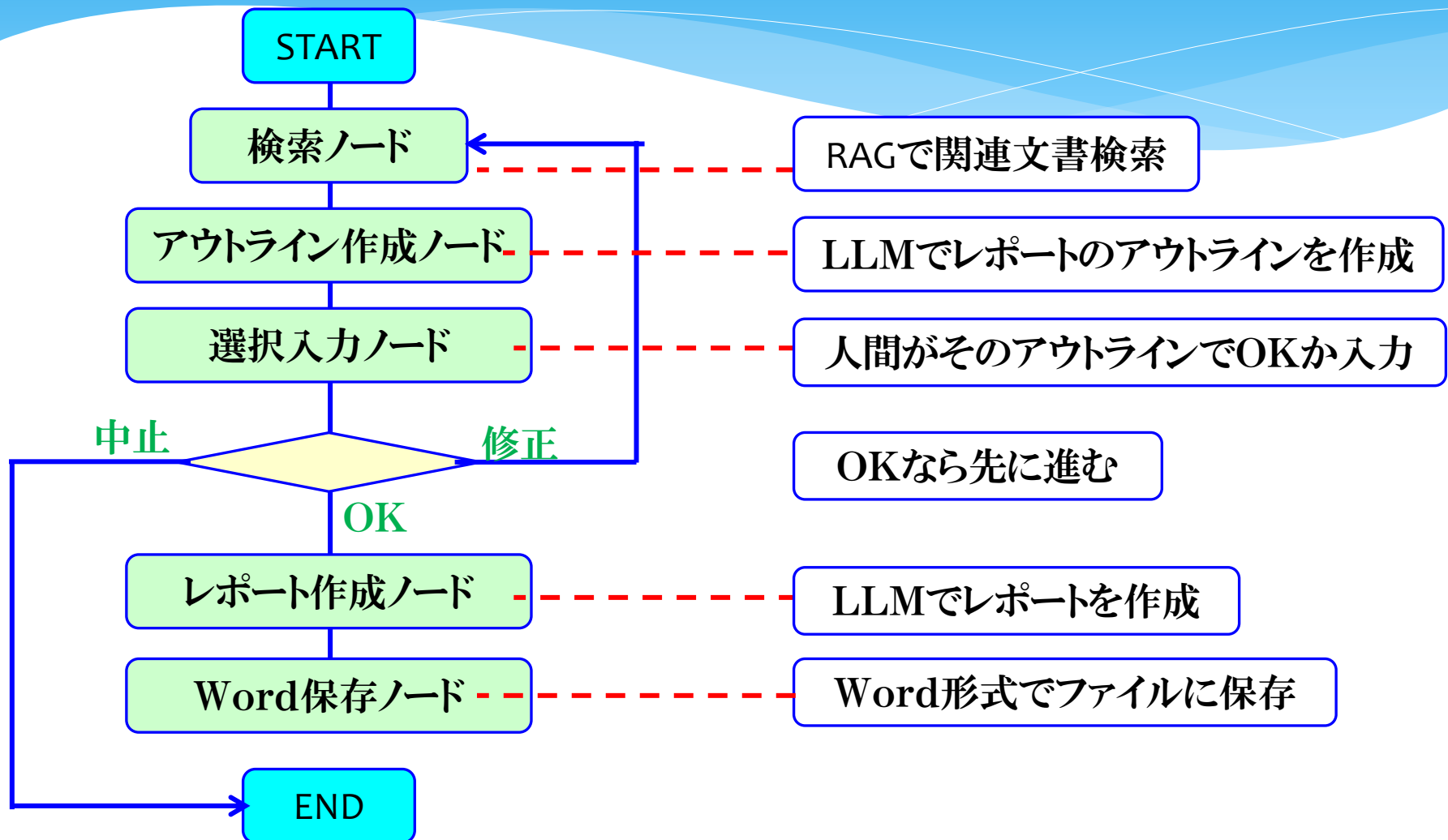
実習① レポート作成アプリ ver1

令和8年7月16日(木)

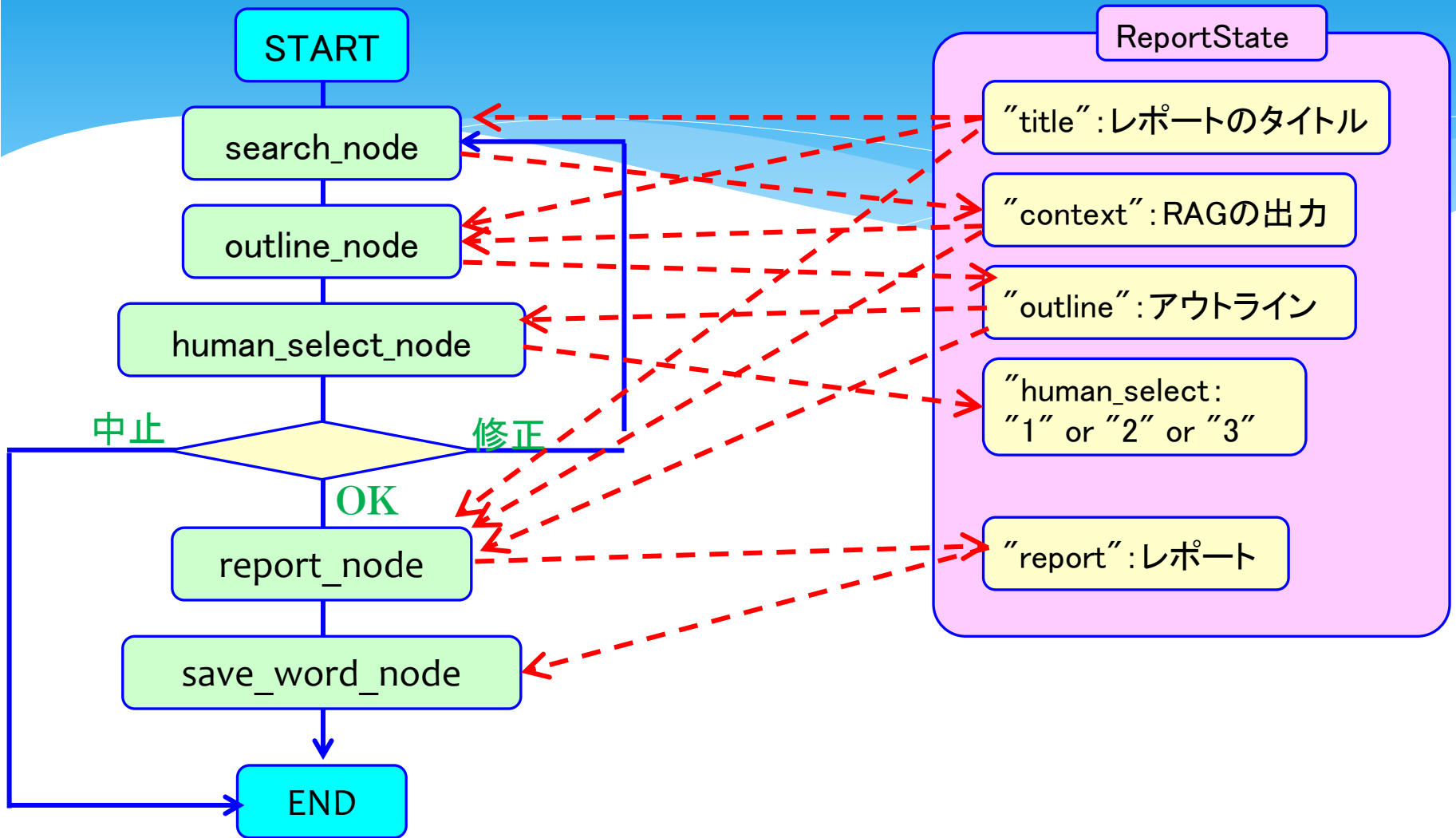
情報・経営システム系 吉田

完成版のワークフロー

エージェントのワークフロー(完成版)

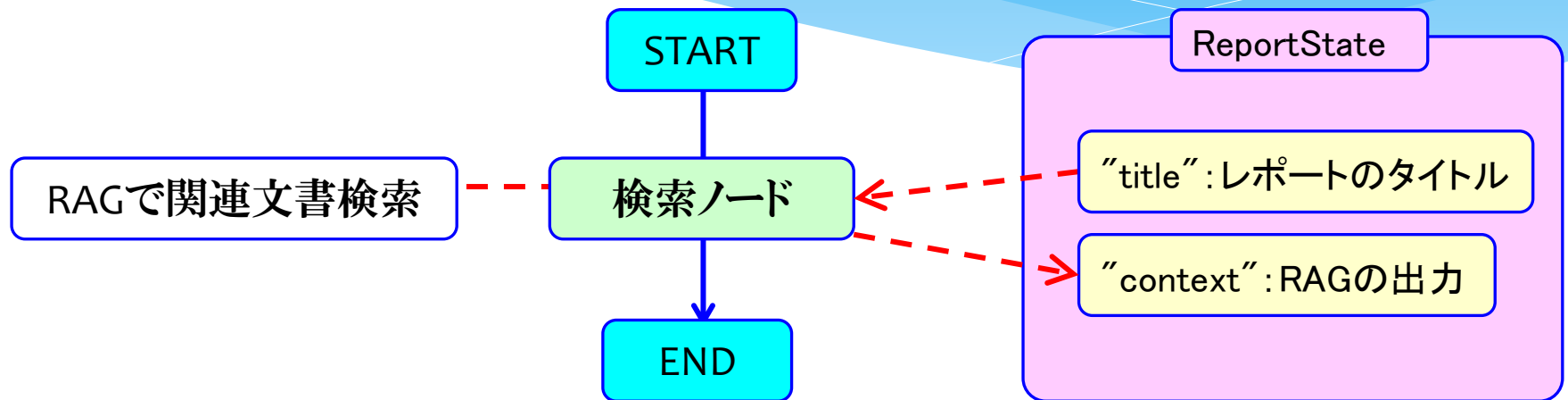


エージェントのワークフローとState (完成版)

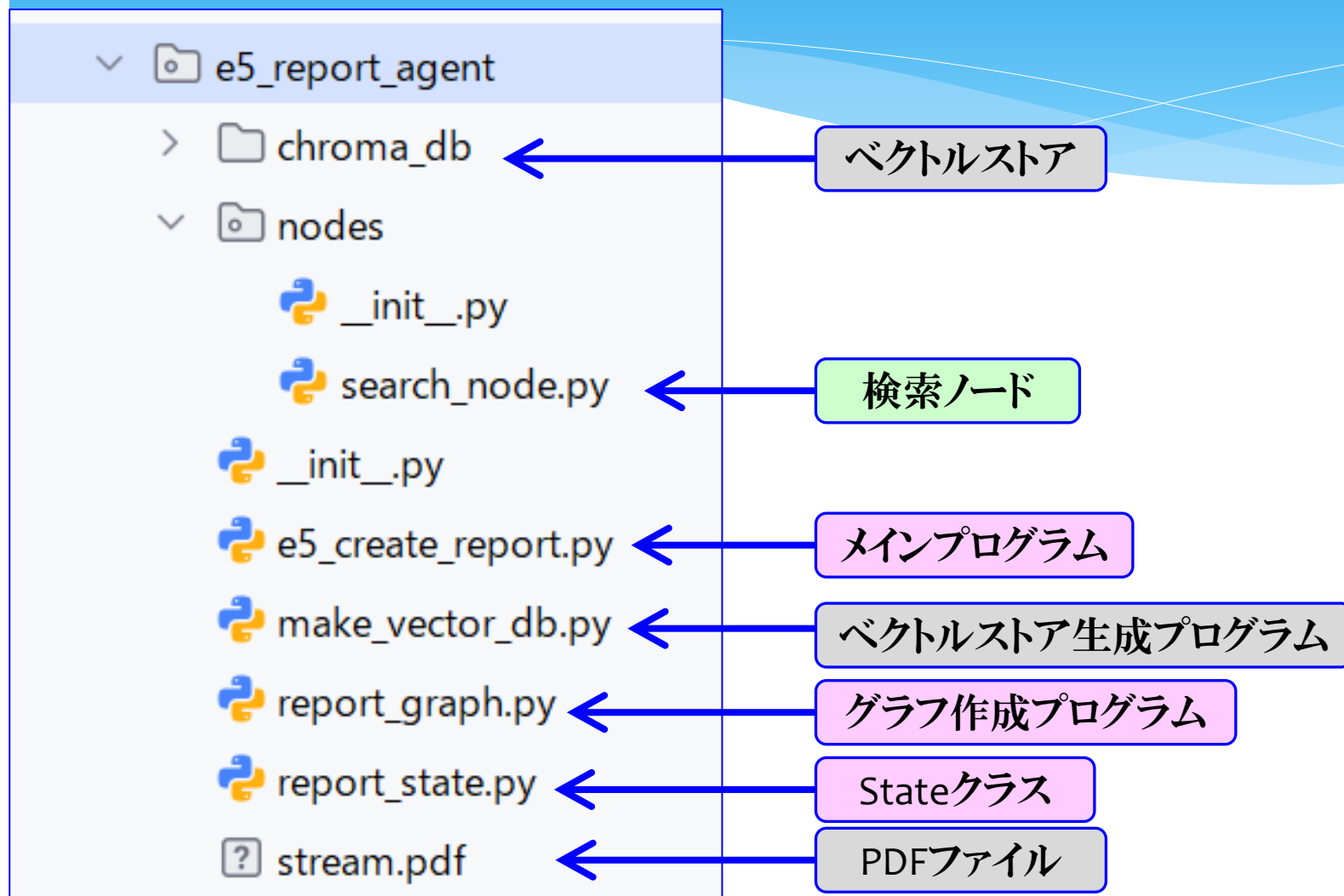


今回のワークフロー

今回のワークフロー

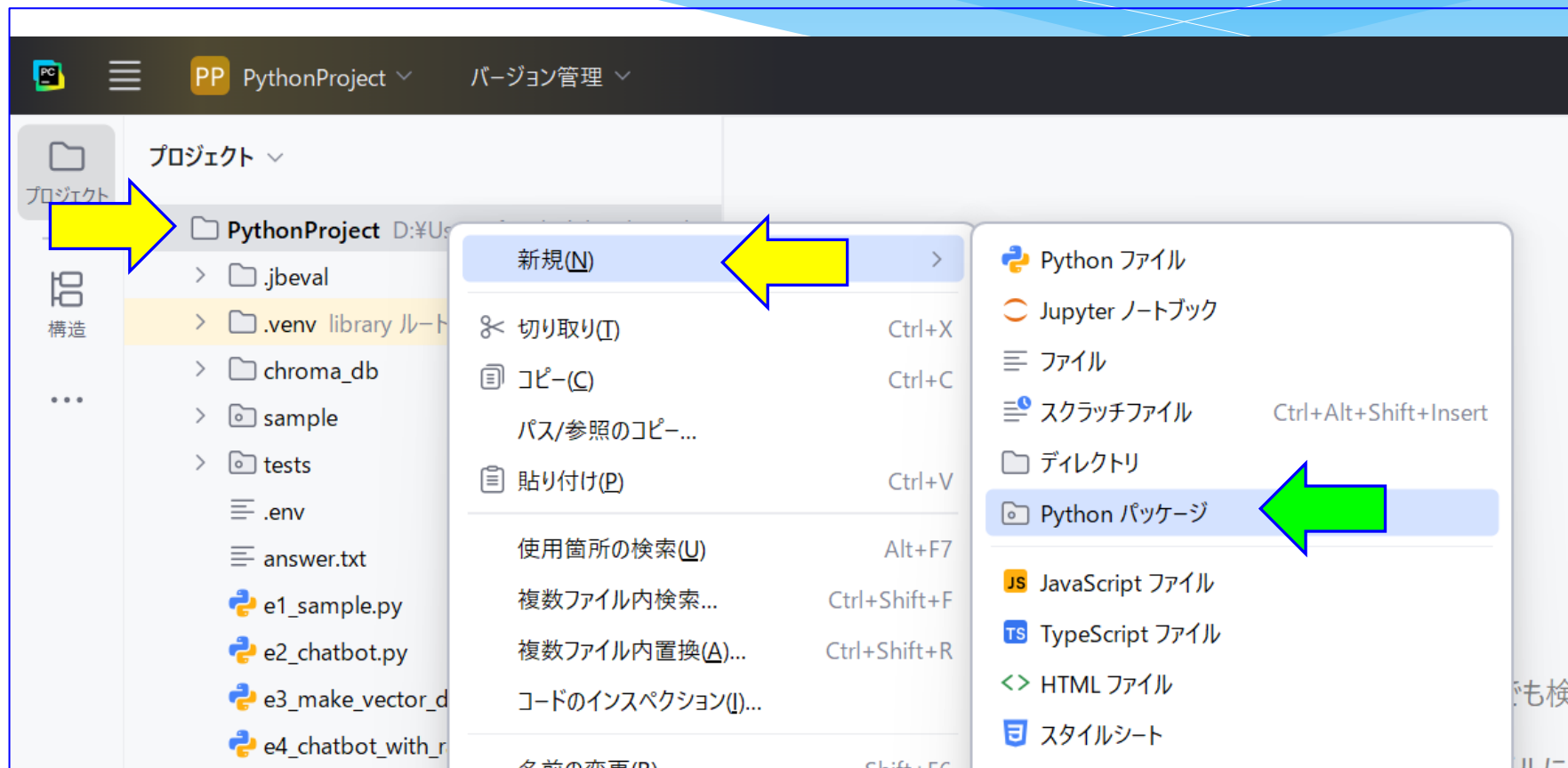


今回はレポート作成アプリをつくることにしよう。
まずはノードが1つだけのエージェントをつくることから始めよう。
最初に配布用フォルダにあるstream.pdfをプロジェクトにコピーしよう。
プロジェクトの構成は以下のようになる。



準備

- ① はじめにパッケージをつくっておこう。
最初は、e5_report_agentパッケージだ。
今回はすべてのプログラムをこのパッケージに入れることにしよう。
PythonProjectで右クリックして、「新規」-「pythonパッケージ」をクリック。

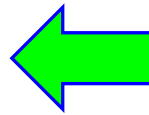


② 「e5_report_agent」と入力してEnter。

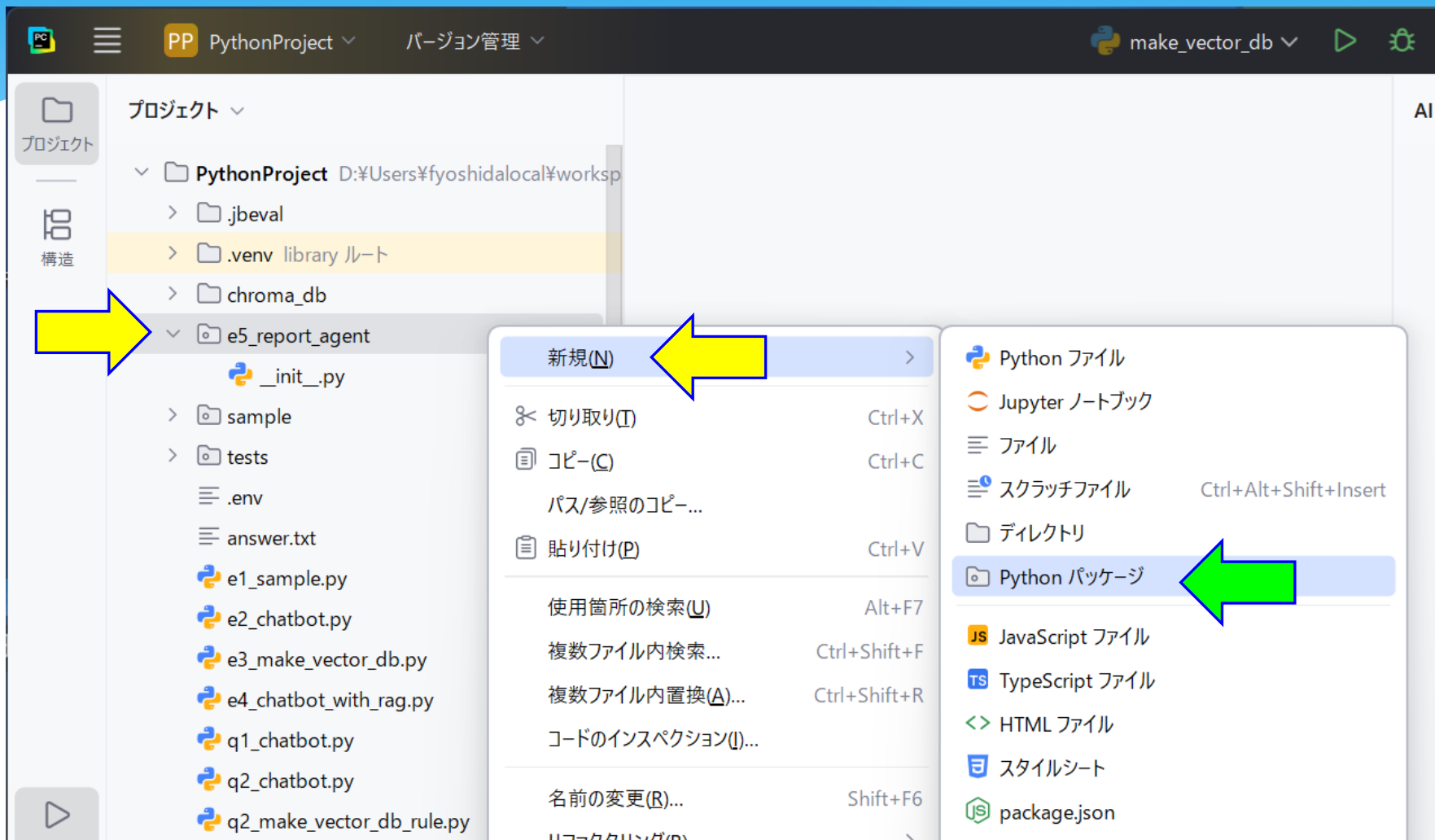
新規 Python パッケージ



e5_report_agent



- ③ 今作成した「e5_report_agent」パッケージで右クリックして、「新規」-「pythonパッケージ」をクリック。

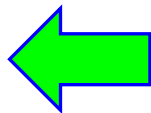


④ 「nodes」と入力してEnter。

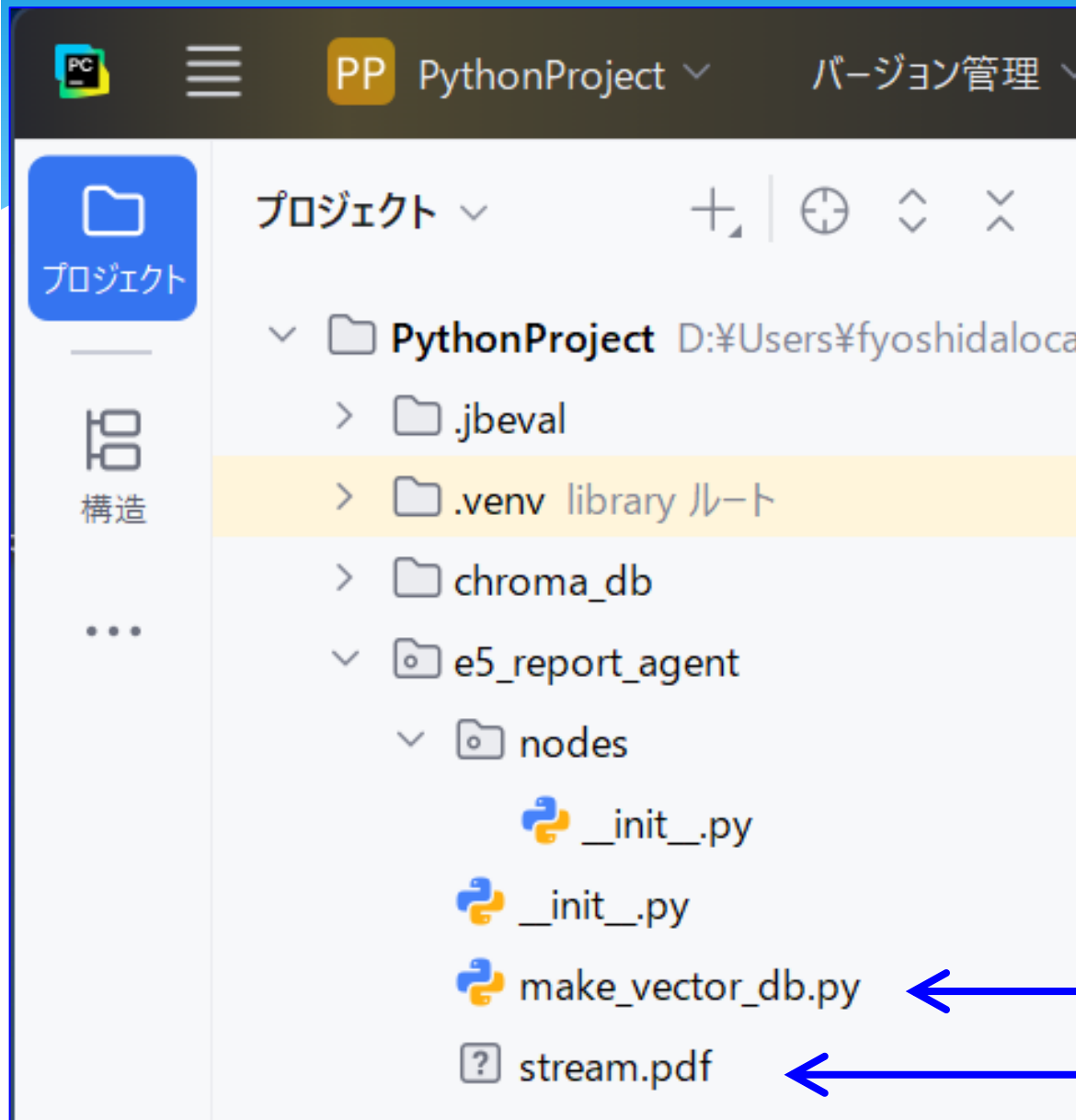
新規 Python パッケージ



nodes|



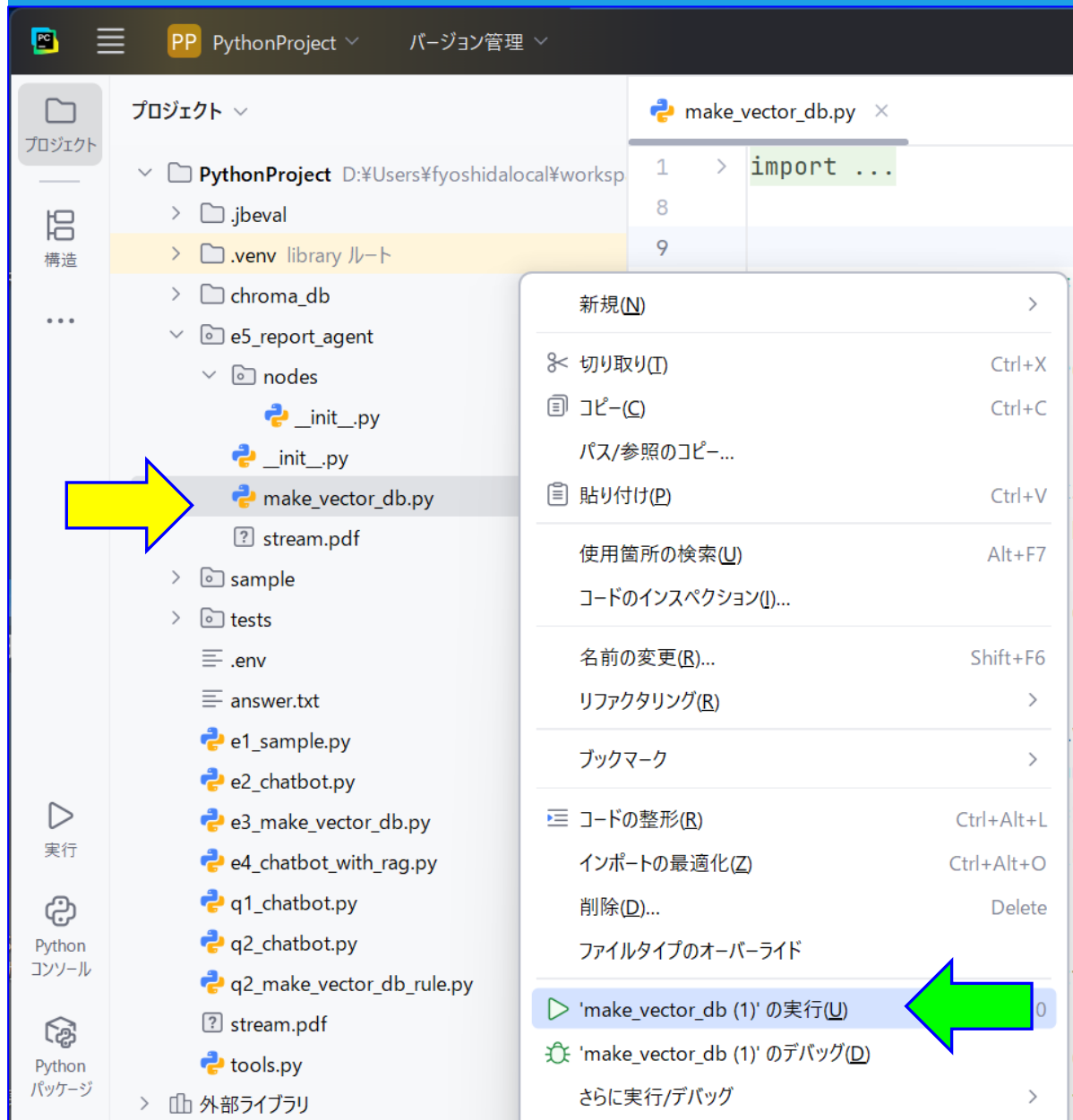
⑤ 前回作成したベクトルストア作成プログラムと、stream.pdfをコピーしてこよう。



ベクトルストア生成プログラム

PDFファイル

⑥ 「make_vector_dv.py」で右クリックして、「make_vector_db.pyの実行」をクリック。



⑦ 「e5_report_agent」パッケージの下にchroma.dbができた！

The screenshot shows the PyCharm IDE interface. On the left, the 'Project' tool window displays the file structure of the 'PythonProject'. The 'e5_report_agent' directory is expanded, showing subdirectories 'chroma_db' and 'nodes', and files '_init_.py' and 'make_vector_db.py'. A green arrow points to the 'chroma_db' directory. The 'make_vector_db.py' file is selected in the editor. The code in the editor is as follows:

```
1 > import ...
8
9
10 PDF_FILE = "stream.pdf"
11 DB_DIR = "./chroma_db"
12 COLLECTION_NAME = "stream_pdf"
13
14 def main():
15     if Path(DB_DIR).exists():
16         shutil.rmtree(DB_DIR)
17
18     loader = PyPDFLoader(PDF_FILE)
```

At the bottom, the 'Terminal' window shows the output of the 'make_vector_db (1)' command. The output includes the file path, connection status, a warning about unauthenticated requests, the progress of loading weights, and the successful creation of the vector database. A green arrow points to the final message: 'プロセスは終了コード 0 で終了しました'.

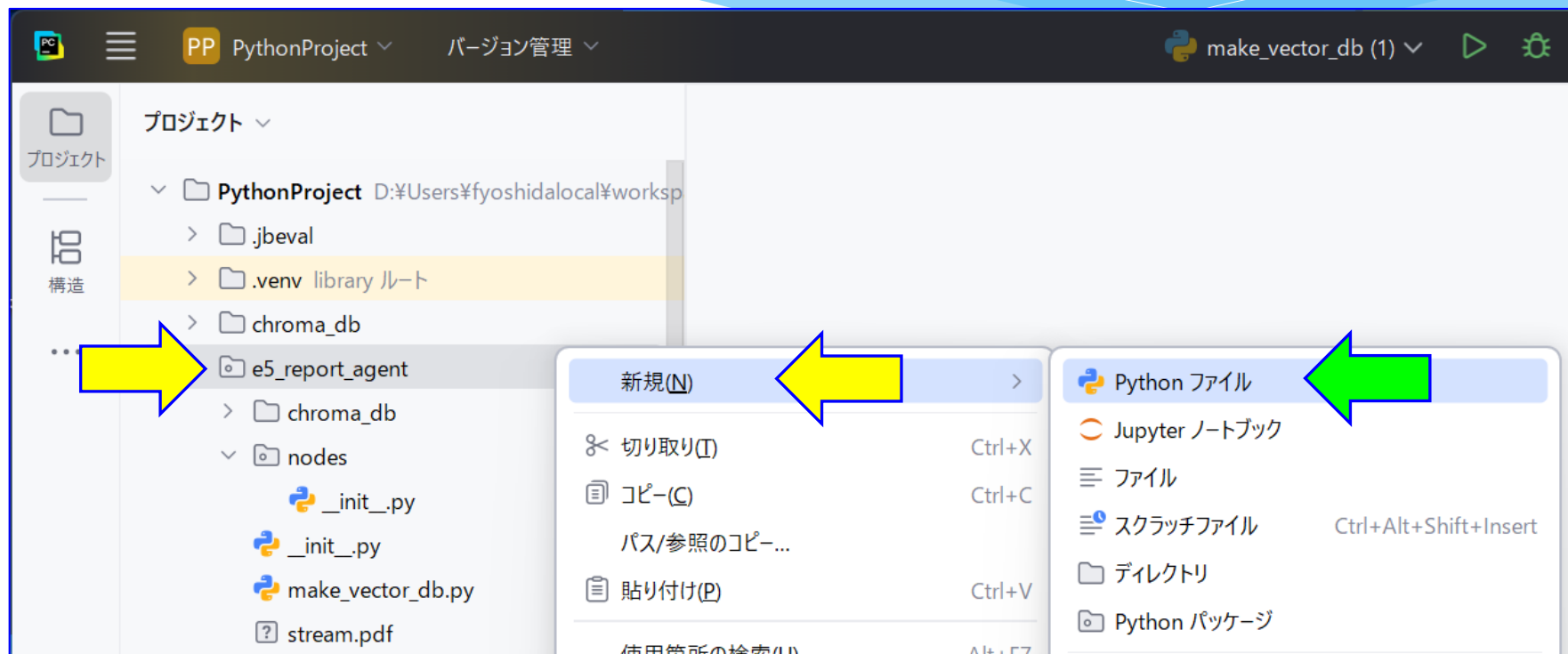
実行 make_vector_db (1) ×

```
D:\Users\fyoshidalocal\workspaces\pycharmprojects\PythonProject\.venv
Connected to server 127.0.0.1:61857
D:\Users\fyoshidalocal\workspaces\pycharmprojects\PythonProject\e5_re
    from langchain_community.document_loaders import PyPDFLoader
Warning: You are sending unauthenticated requests to the HF Hub. Plea
Loading weights: 100%[██████████] 199/199 [00:00<00:00, 6944.67it/s]
ベクトルDBを作成しました。
Disconnected from server

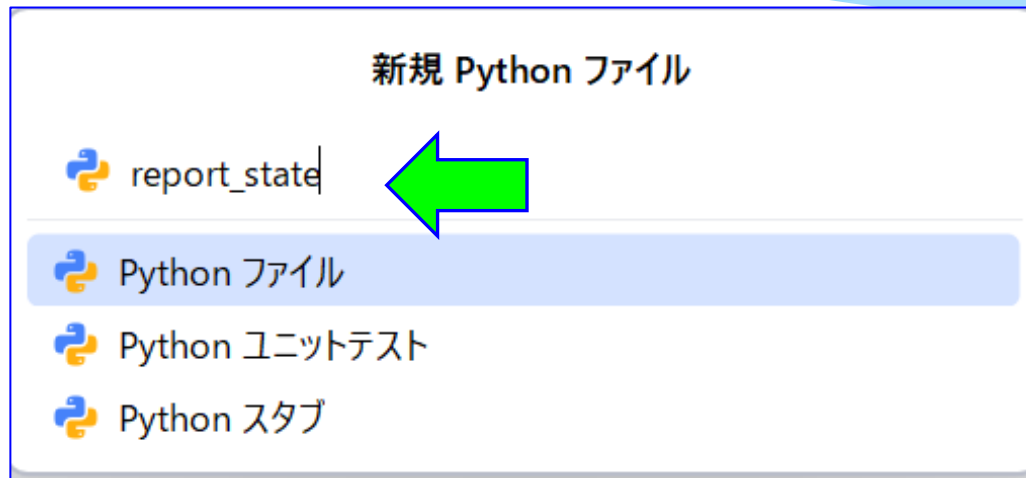
プロセスは終了コード 0 で終了しました
```

Stateの作成

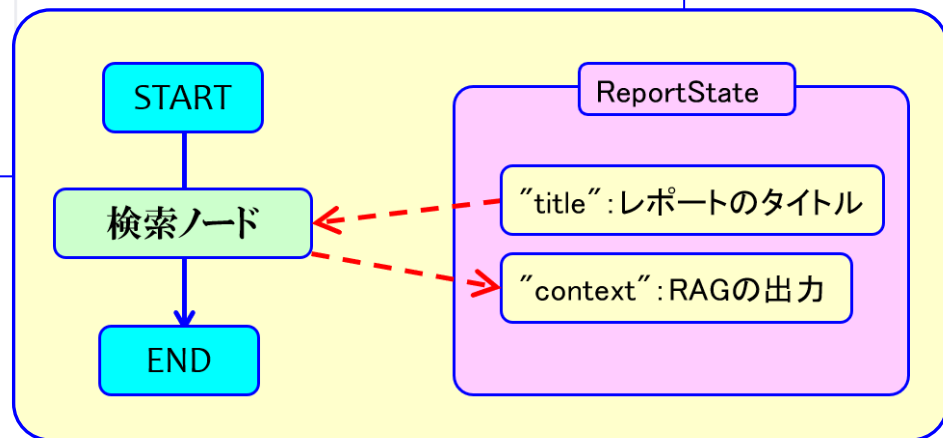
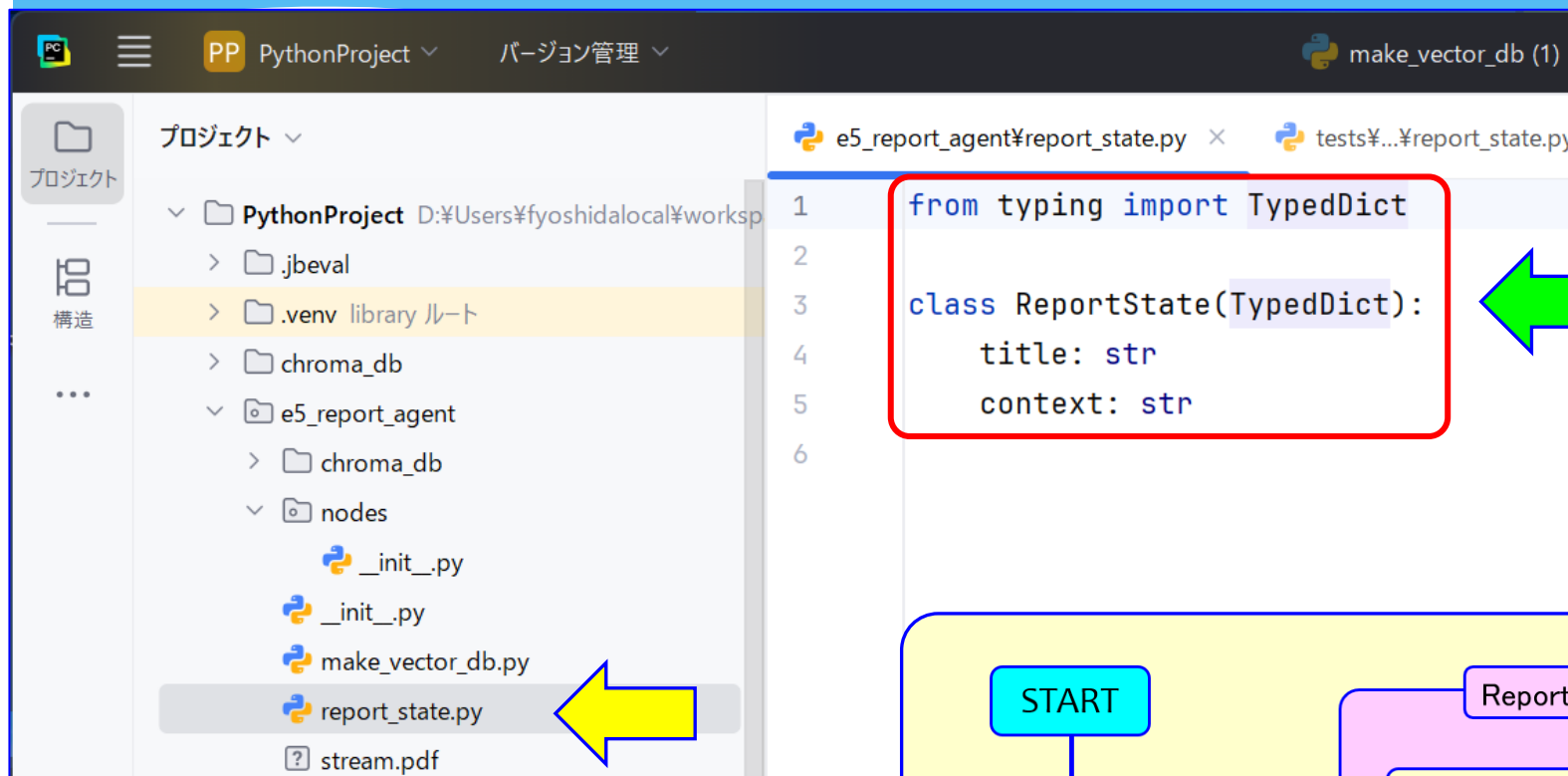
- ① 最初に、ReportStateクラスを作成しよう。
このStateは、ノードへの入力パラメータや、ノードからの戻り値、その他必要な変数を格納するための入れ物だ。
「e5_report_agent」プロジェクトで右クリックして「新規」―「Pythonファイル」をクリック。



② 「report_state」と入力して、Enter。

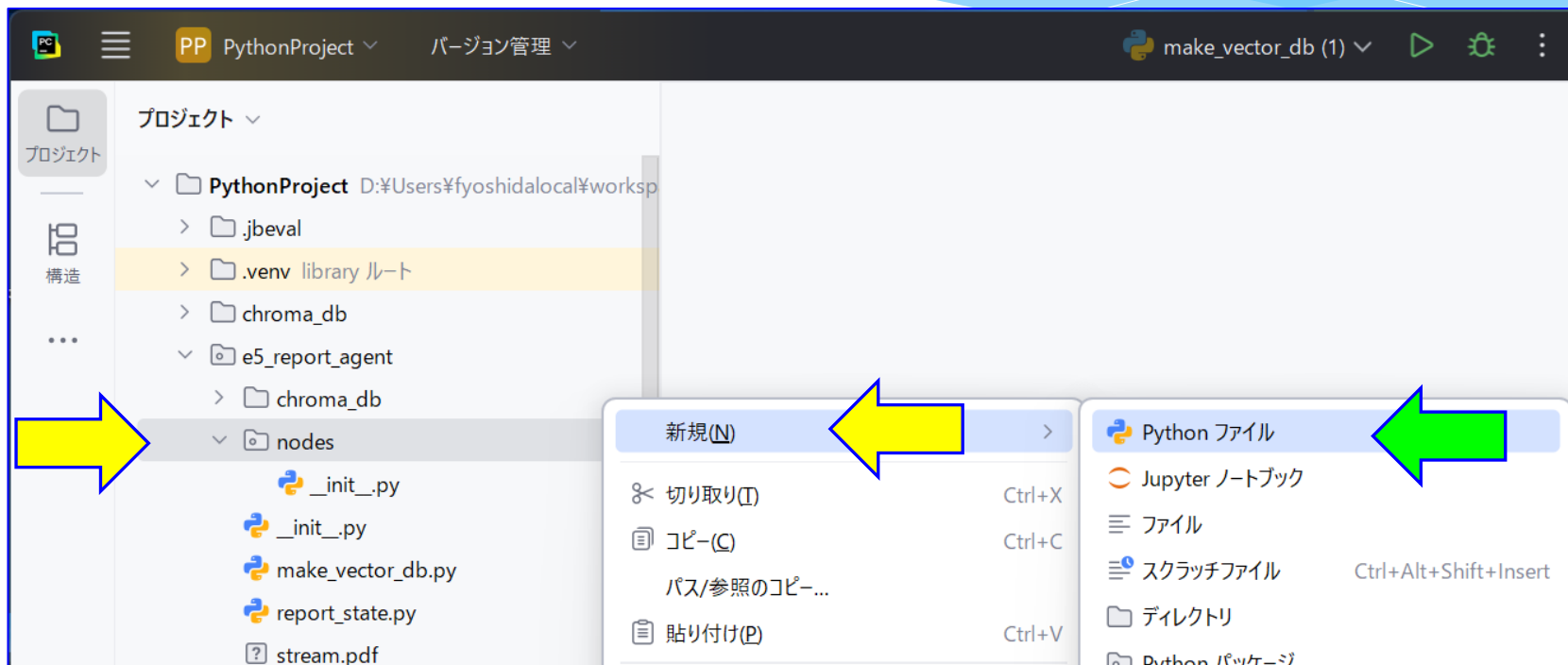


- ③ 今回使用する検索ノードに必要な情報は、
- ・入力パラメータ: "title" (レポートのタイトル)
 - ・戻り値: "context" (Ragで取得したチャンク)
- の2つだけだ。作成されたファイルに以下のように入力しよう、

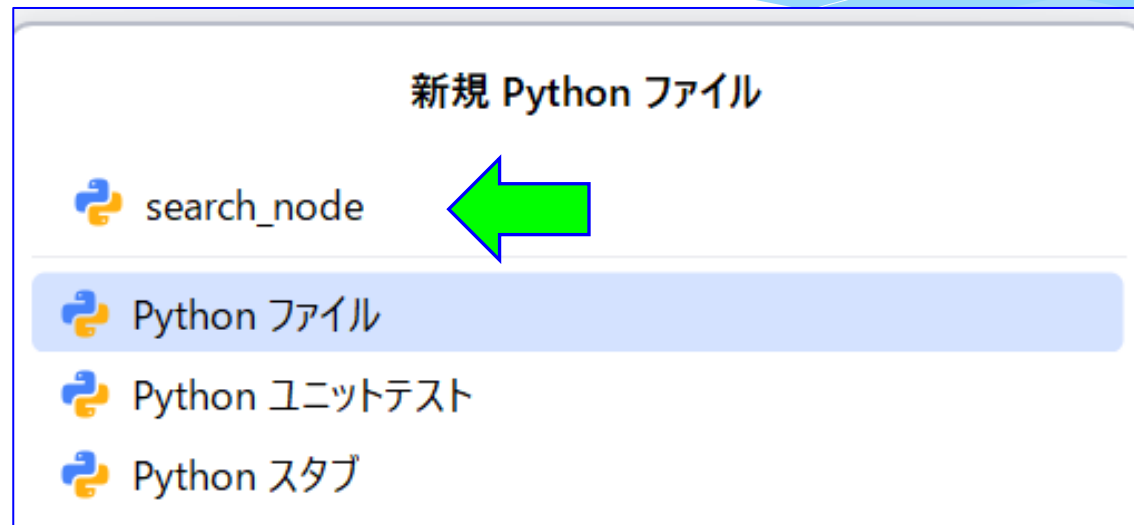


search_nodeの作成

- ① Stateができたので、今度はノードを作成しよう。
「nodes」パッケージで右クリックして、「新規」-「Pythonファイル」をクリック。



② 「search_node」と入力して、Enter。



③ 作成した「search_node.py」に、まずは以下のように入力しよう。
「ノードの処理を行う関数(=search_node本体)は、
「ノードを生成する関数」の中に記述する。

vectorstoreは、エージェント実行中に変化しないので、
nodeを生成する関数の入力パラメータとして与える

search_node
を作る関数

search_node
の処理を行う
関数

必要なライブラリと
ReportStateをimport

ReportStateから、
入力パラメータ“title”
を取り出す

search_nodeが
呼び出されたことを
確認するために表示しておく

ReportStateに格納する
戻り値“context”
をreturnする

```
1 from langchain_chroma import Chroma
2 from e5_report_agent.report_state import ReportState
3
4 def create_search_node(vectorstore: Chroma):
5
6     def search_node(state: ReportState) -> dict:
7         title=state["title"]
8
9         # =====
10        # ここにノードの処理を書く
11        # =====
12
13        print("\n[search_node]")
14
15        return {
16            "context": "",
17        }
18
19    return search_node
```

④ それではsearch_nodeの処理を記述しよう。

```
1 from langchain_chroma import Chroma
2 from e5_report_agent.report_state import ReportState
3
4 def create_search_node(vectorstore: Chroma): 2件以上の使用箇所
5
6     def search_node(state: ReportState) -> dict:
7         title=state["title"]
8
9         retriever = vectorstore.as_retriever(
10             search_kwargs={"k": 3}
11         )
12
13         docs = retriever.invoke(title);
14
15         context_parts = []
16
17         for index, doc in enumerate(docs, start=1):
18             page = doc.metadata.get("page_label", "不明")
19
20             context_parts.append(
21                 f"【資料{index}】 \n"
22                 f"ページ: {page}\n"
23                 f"{doc.page_content[:800]}"
24             )
25
26         print("\n[search_node]")
27         print(f"{len(docs)}件の資料を検索しました。")
28
29         return {
30             "context": "\n\n---\n\n".join(context_parts),
31         }
32
33     return search_node
```

ベクトルストアから
チャンクを3つ取り出す
retrieverを生成

retrieverを使って、
titleに関連するチャンク
を取り出す

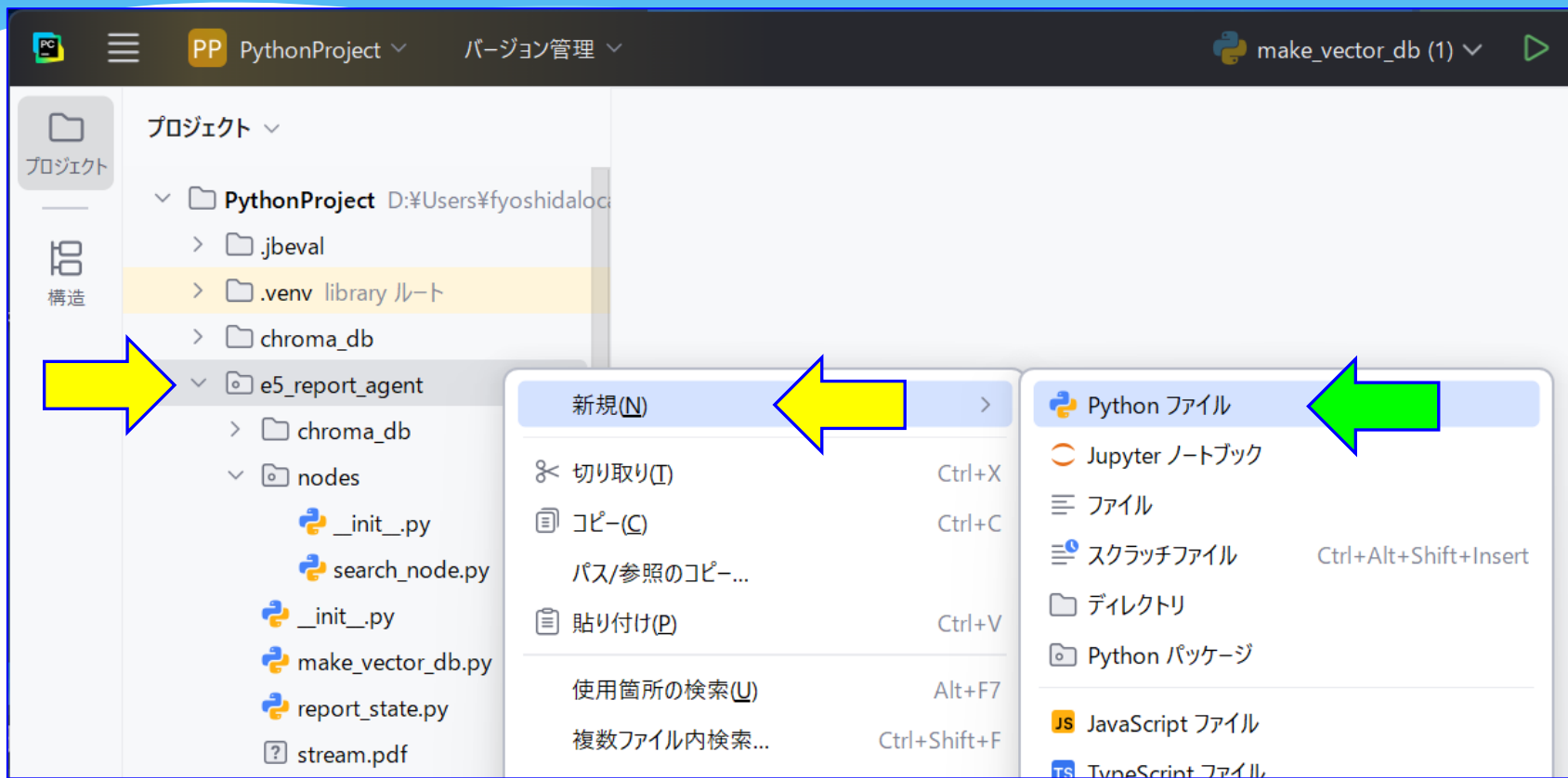
各チャンクに、
indexとpageを追加する

動作確認用に
取り出したチャンク数
を表示しておく

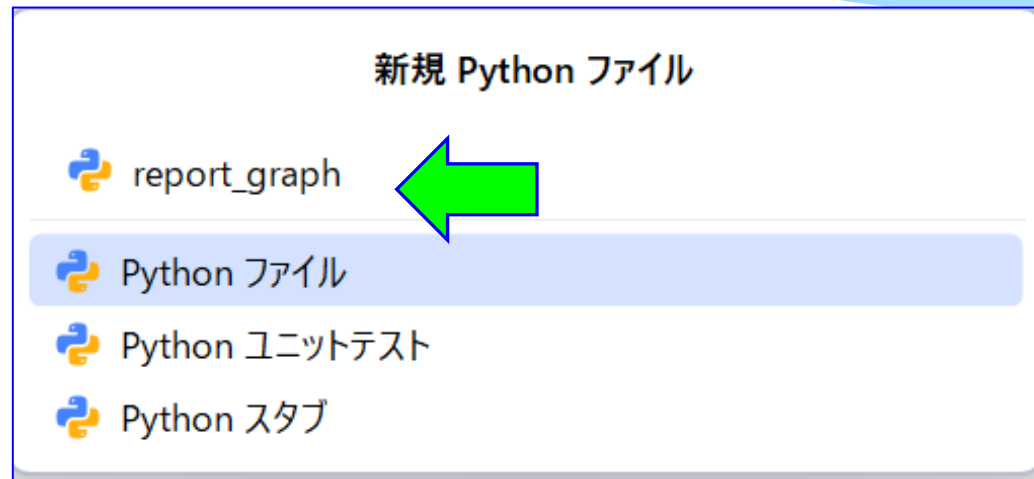
取り出したチャンクにページなどをつけたものを
"context"としてreturnする

graphの作成

- ① Stateとsearch_nodeができたので、
今度はgraph(=ワークフロー)を作成するプログラムreport_graphを作成しよう。
「e5_report_agent」で右クリックして、「新規」-「Pythonファイル」をクリック。



② 「report_graph」と入力して、Enter。



② 作成した「report_graph.py」に以下のように入力しよう。

```
from langchain_chroma import Chroma
from langgraph.graph import StateGraph, START, END
```

必要なライブラリをimport

```
from e5_report_agent.nodes.search_node import create_search_node
from e5_report_agent.report_state import ReportState
```

search_nodeと
ReportStateをimport

```
def create_report_graph(vectorstore: Chroma):
```

```
    builder = StateGraph(ReportState)
```

グラフをつくるためのbuilderを生成

```
    builder.add_node(
        "search",
        create_search_node(vectorstore)
    )
```

① create_search_nodeにvectorstoreを
与えてsearch_nodeをつくる。
② 作ったsearch_nodeを“search”
という名前でbuilderに登録。

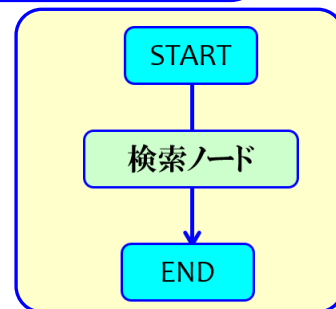
```
    builder.add_edge(START, end_key: "search")
    builder.add_edge(start_key: "search", END)
```

startとsearch_nodeを結ぶエッジと
search_nodeとendを結ぶエッジを
builderに登録



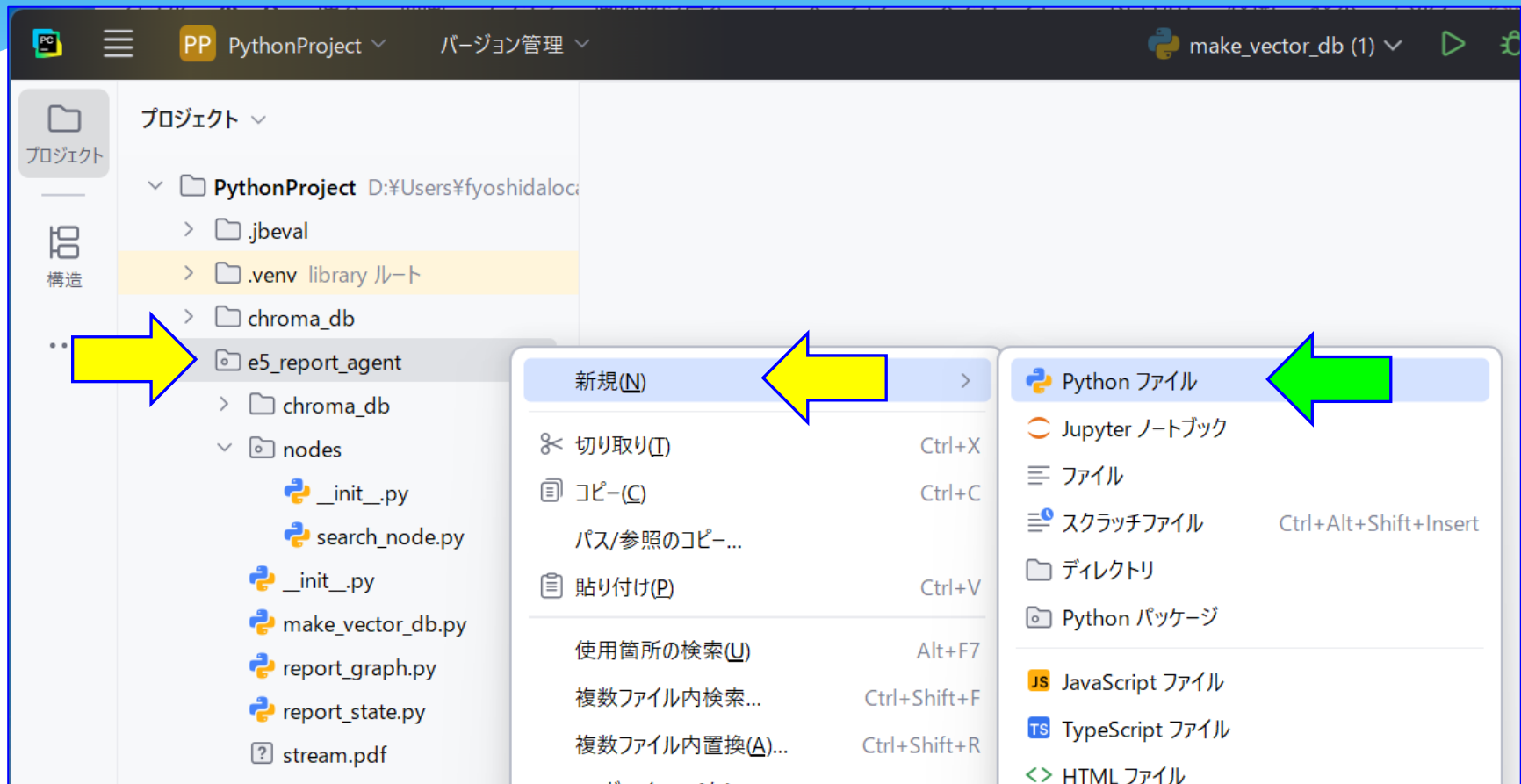
```
    return builder.compile()
```

builderが生成したグラフをreturn



メインプログラムの作成

- ① 最後にメインプログラムを作成しよう。
「e5_report_agent」で右クリックして、「新規」-「Pythonファイル」をクリック。



② 「e5_create_report」と入力して、Enter。

新規 Python ファイル

 e5_create_report|

 Python ファイル

 Python ユニットテスト

 Python スタブ

③ 「e5_create_report.py」に、まずは以下を入力しよう。

```
from pathlib import Path
from dotenv import load_dotenv
```

必要なライブラリをimport

```
|
from langchain_chroma import Chroma
from langchain_huggingface import HuggingFaceEmbeddings
```

埋め込みと
ベクトルストア
をimport

```
from e5_report_agent.report_graph import create_report_graph
from e5_report_agent.report_state import ReportState
```

report_graphと
report_stateを
import

```
load_dotenv()
```

APIキーを読み込む

```
PROJECT_DIR = Path(__file__).resolve().parent
DB_DIR = PROJECT_DIR.parent / "chroma_db"
COLLECTION_NAME = "stream_pdf"
```

ベクトルストアの場所と
ベクトルストアのコレクション名
を設定

```
def main():
```


③ 「e5_create_report.py」に、main()を入力しよう。

```
16 def main():
17
18     embeddings = HuggingFaceEmbeddings(
19         model_name=(
20             "sentence-transformers/"
21             "paraphrase-multilingual-MiniLM-L12-v2"
22         )
23     )
24
25     vectorstore = Chroma(
26         persist_directory=str(DB_DIR),
27         embedding_function=embeddings,
28         collection_name=COLLECTION_NAME
29     )
30
31     graph = create_report_graph(
32         vectorstore=vectorstore
33     )
34
35     title="JavaのStreamについて1000字以内でレポートを作成してください"
36
37     initial_state: ReportState = {
38         "title": title,
39         "context": "",
40     }
41
42     graph.invoke(initial_state)
43
44     print("処理が完了しました")
45
46 if __name__ == "__main__":
47     main()
```

埋め込みツールを生成

ベクトルストア操作ツールを生成

グラフを生成

titleを設定

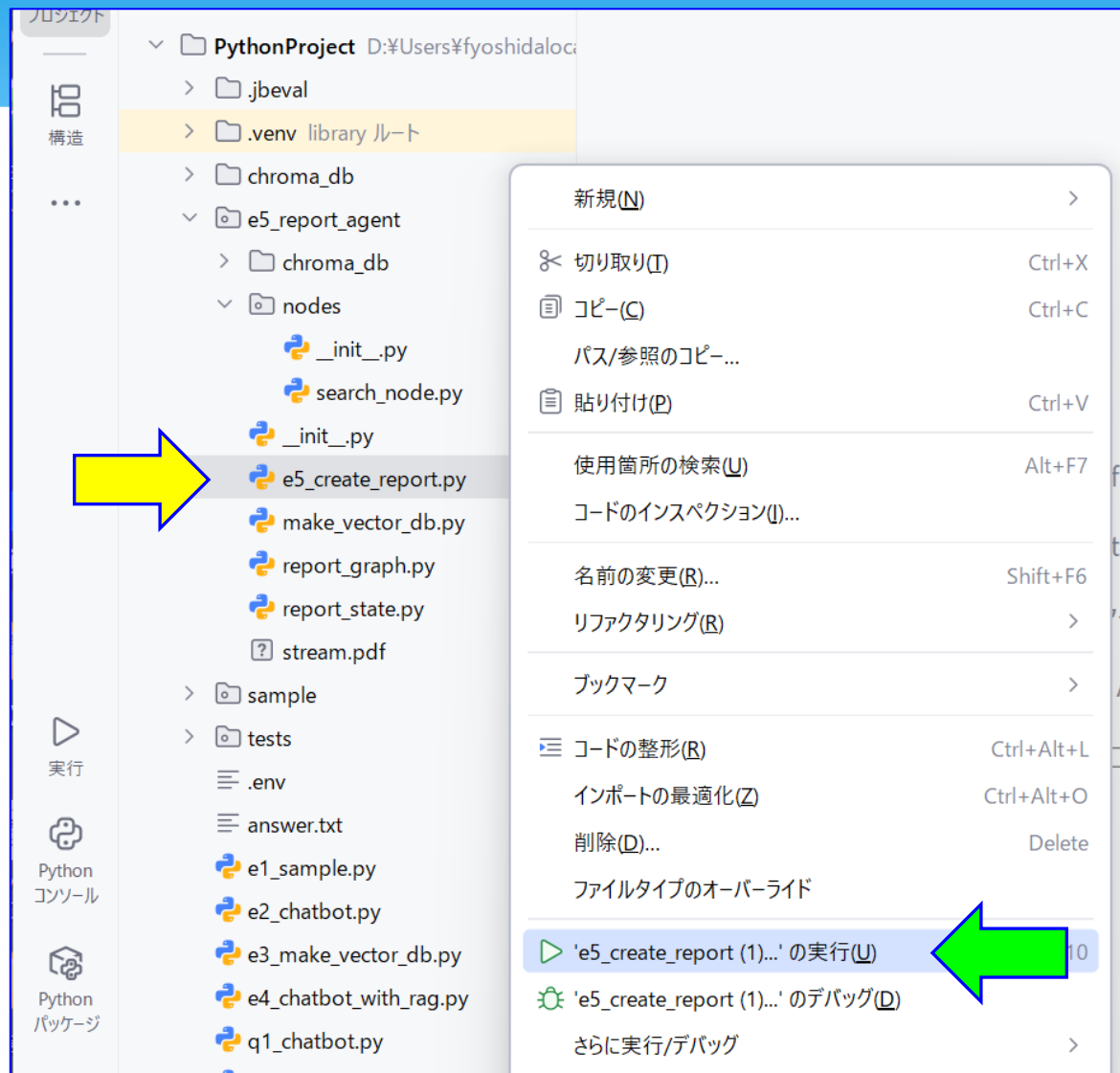
ReportStateの初期状態を設定

初期状態を与えて、グラフを実行

实行

① それでは実行してみよう。

「e5_create_report」で右クリックして、「e5_create_reportの実行」をクリック。



② 以下のように表示されればOKだ！

